

Jolli AI — OSS Repository Health Report

Repo: `jolliai/jolliai` | Scanned: 2026-05-05 | Tool: quaid-scanner v0.1.1 | Depth: quick | Duration: 1,213ms

Executive Summary

Overall Score: 1.7/10 — CRITICAL risk — Maturity: SANDBOX

jolliai is an early-stage repository with a strong foundation in some areas (issue closure rate 85%, Apache-2.0 license present, Code of Conduct, DCO, and 3 issue templates) but significant gaps in security hygiene, community infrastructure, and inclusive language. The project scores 0 across Security, Community, and Inclusive pillars — not because of deep structural problems, but because a small set of well-understood fixes are missing across all three.

The highest-leverage single action is enabling the **OpenSSF Scorecard GitHub Action** — it addresses the security pillar directly and signals supply-chain awareness to adopters. The second is **pinning GitHub Actions to SHA digests**, which alone resolves 13 of the 27 WARNING findings.

Score Summary

Pillar	Score	Bar	Weight
Security	0		25%
Governance	1.5		20%
Community	0		15%
AI Readiness	6.5		15%
Inclusive	0		15%
Technical Rigor	4		10%

Overall weighted score: 1.7/10

Pillar Analysis

Security — 0/10

Security scores 0 because the OpenSSF Scorecard API checks all returned no-signal results (no scorecard badge, no SECURITY.md, no automated security tooling detected) and GitHub Actions workflows have overly-broad token permissions.

Critical findings:

- [CRITICAL]** `token-permissions` : Workflow `".github/workflows/build-intellij.yaml"` has no top-level permissions block (inherits default read-write) (`.github/workflows/build-intellij.yaml`)
 - Suggestion: Add a "permissions:" block with minimal required permissions*
- [CRITICAL]** `token-permissions` : Workflow `".github/workflows/build-vscode.yaml"` has no top-level permissions block (inherits default read-write) (`.github/workflows/build-vscode.yaml`)
 - Suggestion: Add a "permissions:" block with minimal required permissions*
- [CRITICAL]** `token-permissions` : Workflow `".github/workflows/publish-cli.yaml"` has no top-level permissions block (inherits default read-write) (`.github/workflows/publish-cli.yaml`)
 - Suggestion: Add a "permissions:" block with minimal required permissions*

- **[CRITICAL]** `token-permissions` : Workflow ".github/workflows/publish-intellij.yaml" has no top-level permissions block (inherits default read-write) (`.github/workflows/publish-intellij.yaml`)
 - *Suggestion: Add a "permissions:" block with minimal required permissions*
- **[CRITICAL]** `token-permissions` : Workflow ".github/workflows/publish-vscode.yaml" has no top-level permissions block (inherits default read-write) (`.github/workflows/publish-vscode.yaml`)
 - *Suggestion: Add a "permissions:" block with minimal required permissions*

Warning findings:

- `dependency-pinning` (13): Action "actions/checkout" uses major-only version "@v4" in ".github/workflows/build-intell"
 - *Suggestion: Consider pinning to a full semver (e.g., @v4.0.0) or SHA for better reproducibility*
- `openssf-scorecard` (3): Security-Policy check: No SECURITY.md file found
 - *Suggestion: Add a SECURITY.md file with vulnerability reporting instructions*

Recommendations:

1. Add `permissions: read-all` (or least-privilege per-job overrides) to the top level of every workflow file. Affected: `build-intellij.yaml` and at least 4 others. (*impact: high, effort: low*)
2. Add a `SECURITY.md` documenting the vulnerability disclosure process. (*impact: medium, effort: low*)
3. Integrate the [OpenSSF Scorecard Action](#) and add the badge to README. (*impact: high, effort: low*)
4. Pin GitHub Actions to commit SHAs instead of major-version tags (e.g. `actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683` instead of `@v4`). (*impact: high, effort: medium*)

Governance — 1.5/10

Governance has a solid baseline: Apache-2.0 license present and validated, DCO requirement in CONTRIBUTING.md, and a Code of Conduct. Score is held back by missing license headers in source files and a single-vendor contributor distribution.

Critical findings:

- **[CRITICAL]** `vendor-neutrality` : Project is dominated by jolli.ai (100% of commits)
 - *Suggestion: Diversify contributors across multiple organizations to reduce single-vendor risk*

Notable info findings:

- `license-headers` : No SPDX license headers found in 100 source files.
- `binary-artifacts` : Committed JAR detected (`intellij/gradle/wrapper/gradle-wrapper.jar` , 43.7KB). Consider using the Gradle wrapper script only, or document the intentional binary.
- `governance` : No recognizable governance model detected despite governance files being present.

Recommendations:

1. Add SPDX headers (`// SPDX-License-Identifier: Apache-2.0`) to source files. Automate with `reuse` CLI. (*impact: medium, effort: medium*)
2. Add a `GOVERNANCE.md` documenting decision-making process and inviting external contributors to signal the project is open to community ownership. (*impact: medium, effort: low*)
3. Evaluate whether the committed JAR is necessary or can be replaced with a Gradle wrapper script only. (*impact: low, effort: low*)

Community — 0/10

Community scores 0 primarily due to a single contributor in the last 12 months and no funding infrastructure. However, the issue closure rate is an excellent 85% (46 closed / 54 opened in 90 days) — a genuine bright spot that signals maintainer responsiveness.

Warning findings:

- **[WARNING]** `contributor-data` : 1 unique contributor with 1 commits in the last 12 months
 - *Suggestion: Single contributor detected — consider recruiting additional maintainers*
- **[WARNING]** `contributor-funnel` : Conversion rates: casual→regular 0%, regular→core 0%
 - *Suggestion: Low casual-to-regular conversion suggests contributor onboarding friction*
- **[WARNING]** `error` : Scanner "response-classification" failed: Cannot read properties of undefined (reading 're')
 - *Suggestion: Check scanner implementation for errors*
- **[WARNING]** `error` : Scanner "response-time" failed: Cannot read properties of undefined (reading 'repository')
 - *Suggestion: Check scanner implementation for errors*
- **[WARNING]** `support-channels` : No SUPPORT.md or .github/SUPPORT.md found
 - *Suggestion: Add a SUPPORT.md documenting how users can get help*

Notable info findings:

- `bus-factor` : Bus factor 1 — single contributor with 100% of commits in the last 12 months.
- `contributor-funnel` : 0 core, 0 regular, 1 casual contributor. Conversion rate: 0% at every stage.
- `funding` : No FUNDING.yml, GitHub Sponsors, or Open Collective detected.
- `stale-bot` : No stale issue policy — with 85% closure rate this is lower priority.

Recommendations:

1. Add `SUPPORT.md` documenting where users can ask questions. (*impact: medium, effort: low*)
2. Add `FUNDING.yml` linking to a sponsorship channel. (*impact: low, effort: low*)
3. Add a `CONTRIBUTING.md` first-contribution guide to grow the contributor funnel. (*impact: high, effort: medium*)
4. Document or recruit a co-maintainer to reduce single-point-of-failure risk. (*impact: high, effort: high*)

AI Readiness — 6.5/10

AI Readiness is the strongest pillar. Claude Code configuration was detected (PASS), and the welcoming score is 90/100. The repo is not classified as an AI/ML project (no model cards, dataset provenance skipped) which is appropriate for a general-purpose repo.

Passing checks:

- `agentic-rules` : Claude Code configuration detected.
- `welcoming-score` : 90/100 (2 warnings, 4 info items).
- `psych-safety` : CODE_OF_CONDUCT.md present.

Notable warning:

- **[WARNING]** `claude-md-structure` : CLAUDE.md lacks recognized structural sections
 - *Suggestion: Add sections like "Critical Rules", "Project Structure", "Common Tasks" to improve agent guidance.*

Recommendation:

1. Expand CLAUDE.md to include recognized structural sections (project overview, commands, critical rules, environment setup). (*impact: medium, effort: low*)

Inclusive Language — 0/10

Of 41 CRITICAL inclusive findings, 34 are likely false positives for technical terms used in legitimate programming contexts (`abort` ×16, `master` ×11, `whitelist` ×7). The 0 remaining findings warrant review.

Warning findings:

- **[WARNING]** `diminishing-language` : Found diminishing language "obvious" in documentation (`CLAUDE.md`)
 - *Suggestion: Remove "obvious/obviously" — if it were truly obvious, it would not need to be s*
- **[WARNING]** `diminishing-language` : Found diminishing language "obvious" in documentation (`CONTRIBUTING.md`)
 - *Suggestion: Remove "obvious/obviously" — if it were truly obvious, it would not need to be s*

- **[WARNING]** `inclusive-language` : Non-inclusive term "sanity check" found. Consider using: confidence check, validity check, (`RELEASE.md`)
 - `_`Suggestion: Replace "sanity-check" with one of: confidence check, validity check, coherence `_`

Recommendations:

1. Replace `whitelist` → `allowlist` and `sanity check` → `confidence check` / `coherence check` in source files and documentation. (*impact: medium, effort: low*)
2. Replace diminishing language ("obvious", "easy", "simply") in docs with neutral phrasing. (*impact: low, effort: low*)
3. Review `abort` usage — if referring to `AbortController` / `AbortSignal` these are technical standard terms and acceptable. If used as general language, replace with `cancel` or `terminate` . (*impact: low, effort: low*)

Technical Rigor — 4/10

Technical rigor has a reasonable foundation: `package-lock.json` present, 3 issue templates covering bug and feature types, and DCO configured. Held back by no test suite, no linter config, and no releases or SemVer tags.

Critical findings:

- **[CRITICAL]** `test-coverage` : No test files detected in the repository
 - *Suggestion: Add a test suite to improve code reliability and enable coverage tracking*

Warning findings:

- **[WARNING]** `linting` : No linter configuration found
 - *Suggestion: Add a linter (ESLint, Prettier, Ruff, golangci-lint, etc.) and configure it to run in CI*

Notable info findings:

- `release-cadence` : No releases or version tags found — users cannot pin to a stable version.
- `semver` : No git tags. Consider tagging the current HEAD as `v0.1.0` if it is functionally stable.
- `linting` : No ESLint, Ruff, or equivalent config detected.
- `missing-prerequisites` : README contains tool commands but no Prerequisites section.

Recommendations:

1. Add a test suite with ≥80% coverage enforced in CI. (*impact: high, effort: high*)
2. Add ESLint (for TypeScript/JS) or equivalent and run in CI. (*impact: medium, effort: low*)
3. Create an initial `v0.1.0` git tag and publish a GitHub Release. (*impact: medium, effort: low*)
4. Add a Prerequisites section to README. (*impact: low, effort: low*)

What Is Working Well

Check	Finding
<code>dependency-pinning</code>	<code>package-lock.json</code> present with <code>lockfileVersion 3</code>
<code>openssf-scorecard</code>	License check: LICENSE file found
<code>openssf-scorecard</code>	Maintained check: Project has 3 active project indicator files
<code>cla-dco</code>	DCO requirement detected (CONTRIBUTING.md)
<code>license-validation</code>	LICENSE file matches "Apache-2.0" (Apache License, Version 2.0)
<code>license</code>	License detected: Apache-2.0 (confidence: 95%)
<code>issue-closure</code>	Closure ratio: 0.85 (46 closed / 54 opened in 90 days)

psych-safety	Code of Conduct found: CODE_OF_CONDUCT.md
psych-safety	Code of Conduct has enforcement mechanisms (5 signals)
agentic-rules	AI agent configuration detected: Claude Code
welcoming-score	Welcoming score: 90/100 (2 warnings, 4 info)
interaction-templates	Issue templates cover bug and feature types (bug, feature)
interaction-templates	PR template found: .github/pull_request_template.md

Backlog User Stories

Ordered CRITICAL first, then WARNING. One story per actionable category.

Story 1 token-permissions

As a maintainer, I want to add `permissions: read-all` (or scoped per-job permissions) to every GitHub Actions workflow file so that workflows follow least privilege for GITHUB_TOKEN and cannot be used to exfiltrate secrets.

Story 2 vendor-neutrality

As a maintainer, I want to add a GOVERNANCE.md documenting the contribution process and explicitly inviting external maintainers so that the project signals it welcomes community ownership beyond a single vendor.

Story 3 test-coverage

As a developer, I want to add a test suite covering core functionality with a $\geq 80\%$ coverage threshold enforced in CI so that regressions are caught before merge and contributors have a safety net.

Story 4 inclusive-language

As a developer, I want to replace `whitelist` with `allowlist` and `sanity check` with `confidence check` in source and docs so that the codebase aligns with Inclusive Naming Initiative guidelines.

Story 5 dependency-pinning

As a maintainer, I want to pin all GitHub Actions dependencies to commit SHAs instead of mutable version tags so that supply-chain attacks via compromised upstream actions are mitigated.

Story 6 openssf-scorecard

As a maintainer, I want to integrate the OpenSSF Scorecard Action in CI and add the Scorecard badge to README so that supply-chain risk is continuously scored and publicly visible to adopters.

Story 7 support-channels

As a user, I want to add a SUPPORT.md documenting where to ask questions (GitHub Discussions, Discord, email, etc.) so that users know how to get help without filing bug reports for support questions.

Story 8 claude-md-structure

As a developer, I want to expand CLAUDE.md to include standard structural sections (project overview, commands, environment, critical rules) so that AI coding agents have richer context and produce more relevant suggestions for this codebase.

Story 9 diminishing-language

As a contributor, I want to audit documentation for diminishing language ("obvious", "easy") and rewrite with neutral phrasing so that documentation is welcoming to contributors of all experience levels.

Story 10 linting

As a developer, I want to add an ESLint config (for JS/TS) or equivalent and enforce it in CI so that code style is consistent and reviewers can focus on logic rather than formatting.

Story 11 semver

As a maintainer, I want to create an initial `v0.1.0` git tag and set up a GitHub Release workflow so that users can pin to a stable release and track what changes between versions.

Story 12 funding

As a maintainer, I want to add a FUNDING.yml linking to a sponsorship channel (GitHub Sponsors, Open Collective, etc.) so that users can financially support the project's sustainability.

Story 13 contributor-funnel

As a maintainer, I want to add a CONTRIBUTING.md with a step-by-step first-contribution guide so that the barrier for new contributors is lowered and the project can grow beyond a single maintainer.

Story 14 missing-prerequisites

As a contributor, I want to add a Prerequisites section to README listing required tools and minimum versions so that setup friction for new contributors is eliminated.